
PRACTICAL WEEK 1

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

1) Create a file

```
root@DUDDUKURIs: ~/practical/week-2
root@DUDDUKURIs:~# ls
autosync.sh  autosync.sh.save  practical  practical.zip  practical_repo  skill  skill.zip  skill_repo
root@DUDDUKURIs:~# cd practical
root@DUDDUKURIs:~/practical# cd week-2
root@DUDDUKURIs:~/practical/week-2# nano main.c
root@DUDDUKURIs:~/practical/week-2#
```

Then press enter to open command

2) write a code

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
}
```

```

else if (pid == 0) {
    // Child Process
    printf("\nChild Process\n");
    printf("Child PID: %d\n", getpid());

    execl("/bin/sh", "sh", "-c", command, NULL);

    // Executes only if execl fails
    perror("Execution failed");
    exit(1);
}

else {
    // Parent Process
    printf("\nParent Process\n");

```

```

else {
    // Parent Process
    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
}

```

- 1) To save press control O
- 2) To come back to shell press control X
- 3) Then for compilation write a command
gcc filename.c -o filename

```

root@DUDDUKURIs: ~/practical/week-2
root@DUDDUKURIs:~# ls
autosync.sh autosync.sh.save practical practical.zip practic
root@DUDDUKURIs:~# cd practical
root@DUDDUKURIs:~/practical# cd week-2
root@DUDDUKURIs:~/practical/week-2# nano main.c
root@DUDDUKURIs:~/practical/week-2# gcc main.c -o main
root@DUDDUKURIs:~/practical/week-2# █

```

Then press enter then to view a output write a command
./filename

```

root@DUDDUKURIs: ~/practical/week-2
root@DUDDUKURIs:~# ls
autosync.sh autosync.sh.save practical practical.zip practical_re
root@DUDDUKURIs:~# cd practical
root@DUDDUKURIs:~/practical# cd week-2
root@DUDDUKURIs:~/practical/week-2# nano main.c
root@DUDDUKURIs:~/practical/week-2# gcc main.c -o main
root@DUDDUKURIs:~/practical/week-2# ./main
Enter Linux command: █

```

Then you will be asking to enter linux command to view the output
In ENTER LINUX COMMAND: enter either "ls" command (or) "pwd" command
to see the output of your code

```

root@DUDDUKURIs:~/practical/week-2# nano main.c
root@DUDDUKURIs:~/practical/week-2# gcc main.c -o main
root@DUDDUKURIs:~/practical/week-2# ./main
Enter Linux command: ls

Parent PID: 607
Child PID : 610

Child PID : 610
Parent PID: 607

Executing command: ls
main main.c

Child process completed.
root@DUDDUKURIs:~/practical/week-2# █

```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```

root@DUDDUKURIs:~# uname
Linux
root@DUDDUKURIs:~# █

```

```

root@DUDDUKURIs:~# lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 39 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 16
On-line CPU(s) list: 0-15
Vendor ID: GenuineIntel
Model name: 13th Gen Intel(R) Core(TM) i7-13620H
CPU family: 6
Model: 186
Thread(s) per core: 2
Core(s) per socket: 8
Socket(s): 1
Stepping: 2
BogoMIPS: 5836.82
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable
monitor_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowpre
ferth_ssb ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust hle avx2 smep bmi2 erms invpcid rdseed adx cswap clflushopt clwb sha_ni xsaveopt xsavec vgetbv
1 xsave avx_vnni vmiai umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b frm md_clear serialize_ibt flush_l1d arch_capabilities
Virtualization features:
Virtualization: VT-x
Hypervisor vendor: Microsoft
Virtualization type: full
Caches (sum of all):
L1d: 384 KiB (8 instances)
L1i: 256 KiB (8 instances)
L2: 10 MiB (8 instances)
L3: 24 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-15

```

```

Vulnerabilities:
Gather data sampling: Not affected
Ghostwrite: Not affected
Indirect target selection: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Vulnerable; No microcode
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI BHI_DIS_S
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected

```

```

root@DUDDUKURIs:~# ps
  PID TTY          TIME CMD
  639 pts/0        00:00:00 bash
  665 pts/0        00:00:00 ps
root@DUDDUKURIs:~#

```

```

root@DUDDUKURIs:~#
top - 03:13:54 up 20 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.9 id,  0.1 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  7787.7 total,  7182.6 free,   556.3 used,   200.4 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used,  7231.4 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
228	syslog	20	0	222516	5984	4584	S	0.3	0.1	0:00.08	rsyslogd
697	root	20	0	9416	5732	3488	R	0.3	0.1	0:00.01	top
1	root	20	0	21732	13196	9788	S	0.0	0.2	0:01.23	systemd
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.03	init-systemd(Ub
7	root	20	0	3212	2140	2008	S	0.0	0.0	0:00.02	init
59	root	19	-1	50452	15892	14788	S	0.0	0.2	0:00.32	systemd-journal
106	root	20	0	25108	6544	5168	S	0.0	0.1	0:00.25	systemd-udev
115	systemd+	20	0	21344	13232	11108	S	0.0	0.2	0:00.09	systemd-resolve
116	systemd+	20	0	91036	7992	7024	S	0.0	0.1	0:00.08	systemd-timesyn
185	root	20	0	4244	2684	2432	S	0.0	0.0	0:00.00	cron
186	message+	20	0	9596	5396	4712	S	0.0	0.1	0:00.10	dbus-daemon
200	root	20	0	17976	8836	7796	S	0.0	0.1	0:00.07	systemd-logind
211	root	20	0	1755848	11532	9224	S	0.0	0.1	0:00.11	wsl-pro-service
239	root	20	0	3124	1976	1836	S	0.0	0.0	0:00.00	agetty
251	root	20	0	107032	23404	13952	S	0.0	0.3	0:00.12	unattended-upgr
332	root	20	0	6672	4564	3796	S	0.0	0.1	0:00.01	login
378	root	20	0	20324	11576	9452	S	0.0	0.1	0:00.14	systemd
379	root	20	0	21172	3512	1796	S	0.0	0.0	0:00.00	(sd-pam)
402	root	20	0	6080	5296	3652	S	0.0	0.1	0:00.02	bash
631	root	20	0	3184	1112	980	S	0.0	0.0	0:00.00	SessionLeader
632	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.00	Relay(639)
639	root	20	0	6076	5368	3668	S	0.0	0.1	0:00.03	bash